

04_alerts-js

#javascript #frontend #api #fastapi #html #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ frontend/  
    └─ assets/  
        └─ alerts.js
```

El archivo `alerts.js` se encuentra dentro de:

```
frontend/assets/
```

Este archivo contiene la lógica JavaScript específica de la página principal de alertas:

```
frontend/index.html
```

Su función es cargar alertas desde la API, aplicar filtros, gestionar paginación, sincronizar el estado con la URL y pintar dinámicamente la tabla HTML.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,420p' frontend/assets/alerts.js
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Permite visualizar contenido de archivos.

```
-n
```

Evita que `sed` imprima todo automáticamente.

```
'1,420p'
```

Imprime desde la línea 1 hasta la 420.

```
frontend/assets/alerts.js
```

Ruta del archivo analizado.

3 Código completo del archivo

```
let state = {  
  limit: 50,  
  offset: 0,  
  status: "",  
  group_key: "",  
};
```

```

function readStateFromUrl() {
  const u = new URL(window.location.href);
  state.limit = Number(u.searchParams.get("limit") ?? "50");
  state.offset = Number(u.searchParams.get("offset") ?? "0");
  state.status = u.searchParams.get("status") ?? "";
  state.group_key = u.searchParams.get("group_key") ?? "";
}

function syncForm() {
  qs("limit").value = String(state.limit);
  qs("status").value = state.status;
  qs("group_key").value = state.group_key;
}

function syncUrl() {
  setQueryParams({
    limit: state.limit,
    offset: state.offset,
    status: state.status,
    group_key: state.group_key,
  });
}

function renderRows(alerts) {
  const tb = qs("alertsTbody");
  tb.innerHTML = "";

  if (!alerts || alerts.length === 0) {
    const tr = document.createElement("tr");
    tr.innerHTML = `<td colspan="9" class="muted">Sin resultados</td>`;
    tb.appendChild(tr);
    return;
  }

  for (const a of alerts) {
    const tr = document.createElement("tr");
    tr.innerHTML = `
      <td class="mono">${a.id}</td>
      <td><span class="${statusBadgeClass(a.status)}">${a.status}</span></td>
      <td class="muted"><!--</td>
      <td class="mono">${a.group_key ?? "--}</td>
      <td>${escapeHtml(a.title)}</td>
      <td class="mono">${a.rule_id}</td>
      <td class="mono">${a.event_id}</td>
      <td class="mono">${fmtDate(a.created_at)}</td>
      <td><a class="btn" href="/alert.html?id=${encodeURIComponent(a.id)}">Ver</a></td>
    `;
    tb.appendChild(tr);
  }
}

function escapeHtml(s) {
  return String(s ?? "")
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;")
    .replaceAll('"', "&#039;");
}

async function loadAlerts() {
  const err = qs("errorBox");
  const info = qs("infoBox");
  hide(err); hide(info);
}

```

```

qs("alertsTbody").innerHTML = `<tr><td colspan="9" class="muted">Cargando...</td></tr>`;

try {
  const data = await apiFetch("/alerts", {
    query: {
      limit: state.limit,
      offset: state.offset,
      status: state.status || null,
      group_key: state.group_key || null,
    }
  });

  renderRows(data);

  qs("resultMeta").textContent =
    `limit=${state.limit} · offset=${state.offset} · resultados=${data.length}`;

  // UX: si vuelve vacío y offset>0, probablemente te pasaste de página.
  if (data.length === 0 && state.offset > 0) {
    show(info, "No hay más resultados en esta página. Prueba con Prev.");
  }

  qs("btnPrev").disabled = state.offset <= 0;
  // Next: no sabemos total; habilitamos siempre. Si vacía, el mensaje guía.
  qs("btnNext").disabled = false;

} catch (e) {
  show(err, `Error cargando alertas: ${e.message}`);
}

function init() {
  readStateFromUrl();
  syncForm();

  qs("filtersForm").addEventListener("submit", (ev) => {
    ev.preventDefault();
    state.status = qs("status").value.trim();
    state.group_key = qs("group_key").value.trim();
    state.limit = Number(qs("limit").value);
    state.offset = 0; // aplicar filtros resetea paginación
    syncUrl();
    loadAlerts();
  });

  qs("btnClear").addEventListener("click", () => {
    state.status = "";
    state.group_key = "";
    state.offset = 0;
    syncForm();
    syncUrl();
    loadAlerts();
  });

  qs("btnPrev").addEventListener("click", () => {
    state.offset = Math.max(0, state.offset - state.limit);
    syncUrl();
    loadAlerts();
  });

  qs("btnNext").addEventListener("click", () => {
    state.offset = state.offset + state.limit;
    syncUrl();
    loadAlerts();
  });
}

```

```
qs("btnRefresh").addEventListener("click", () => loadAlerts());

loadAlerts();
}

document.addEventListener("DOMContentLoaded", init);
```

4 Función general del archivo

alerts.js controla la página principal de alertas.

Es el archivo que convierte index.html en una página dinámica.

Sus responsabilidades principales son:

- Mantener el estado actual de filtros y paginación.
- Leer filtros desde la URL.
- Sincronizar el formulario HTML con el estado interno.
- Sincronizar la URL con el estado interno.
- Llamar al backend mediante GET /alerts.
- Renderizar las alertas recibidas en la tabla.
- Gestionar mensajes de carga, error e información.
- Gestionar los botones Aplicar, Limpiar, Prev, Next y Actualizar.

La relación principal es:

```
index.html
  ↓
define tabla, filtros y botones

alerts.js
  ↓
lee controles HTML
  ↓
llama a GET /alerts
  ↓
rellena la tabla
```

5 Estructura general del archivo

El archivo se puede dividir en ocho bloques:

```
let state = { ... };
```

Estado actual de la página.

```
function readStateFromUrl() { ... }
```

Lee filtros y paginación desde la URL.

```
function syncForm() { ... }
```

Actualiza el formulario con los valores del estado.

```
function syncUrl() { ... }
```

Actualiza la URL con los valores del estado.

```
function renderRows(alerts) { ... }
```

Renderiza las alertas dentro de la tabla.

```
function escapeHtml(s) { ... }
```

Escapa texto antes de insertarlo en HTML.

```
async function loadAlerts() { ... }
```

Carga alertas desde la API.

```
function init() { ... }
```

Inicializa eventos, estado y primera carga.

```
document.addEventListener("DOMContentLoaded", init);
```

Ejecuta la inicialización cuando el DOM está listo.

Visualmente:

```
alerts.js
├─ state
├─ readStateFromUrl()
├─ syncForm()
├─ syncUrl()
├─ renderRows()
├─ escapeHtml()
├─ loadAlerts()
├─ init()
└─ DOMContentLoaded
```

6 Análisis línea por línea

Estado inicial

```
let state = {
  limit: 50,
  offset: 0,
  status: "",
  group_key: "",
};
```

Define un objeto llamado `state`.

Este objeto guarda el estado actual de la vista principal.

Campos:

```
limit    → número máximo de alertas a cargar
offset   → desplazamiento para paginación
status   → filtro por estado
group_key → filtro por grupo/host
```

let state

```
let state
```

Usa `let` porque el objeto `state` puede modificarse durante la ejecución.

Aunque no se reasigne el objeto completo, sus propiedades sí cambian.

limit: 50

```
limit: 50,
```

Define que por defecto se cargarán 50 alertas.

Esto coincide con el `select` de `index.html`:

```
<option value="50" selected>50</option>
```

Y con el backend, donde `GET /alerts` tiene:

```
limit: int = Query(50, ge=1, le=500)
```

offset: 0

```
offset: 0,
```

Indica que la primera carga empieza desde el primer resultado.

Se usa para paginación.

Ejemplo:

```
offset = 0    → primera página
offset = 50   → segunda página si limit=50
offset = 100  → tercera página si limit=50
```

status: ""

```
status: "",
```

Filtro de estado vacío.

Esto significa que inicialmente se muestran alertas de cualquier estado:

```
open
ack
closed
```

group_key: ""

```
group_key: "",
```

Filtro de `group_key` vacío.

Esto significa que inicialmente no se filtra por host o grupo.

Función `readStateFromUrl`

```
function readStateFromUrl() {
```

Define una función que lee los parámetros actuales de la URL y los guarda en `state`.

Esto permite que la página recuerde filtros al recargar o compartir una URL.

Crear URL actual

```
const u = new URL(window.location.href);
```

Crea un objeto `URL` usando la dirección actual del navegador.

Ejemplo:

```
http://localhost:5173/index.html?limit=25&offset=50&status=open&group_key=server-01
```

A partir de ese objeto se pueden leer los parámetros.

Leer `limit`

```
state.limit = Number(u.searchParams.get("limit") ?? "50");
```

Lee el parámetro `limit` de la URL.

Desglose:

```
u.searchParams.get("limit")
```

Obtiene el valor de `limit`.

Si no existe, devuelve `null`.

```
?? "50"
```

Si el valor es `null` o `undefined`, usa `"50"`.

```
Number(...)
```

Convierte el resultado a número.

Ejemplos:

```
?limit=25 → state.limit = 25  
sin limit → state.limit = 50
```

Leer `offset`

```
state.offset = Number(u.searchParams.get("offset") ?? "0");
```

Lee el desplazamiento de paginación.

Si no existe, usa 0.

Ejemplos:

```
?offset=50 → state.offset = 50  
sin offset → state.offset = 0
```

Leer `status`

```
state.status = u.searchParams.get("status") ?? "";
```

Lee el estado desde la URL.

Si no existe, usa cadena vacía.

Ejemplos:

```
?status=open → state.status = "open"
sin status    → state.status = ""
```

Leer `group_key`

```
state.group_key = u.searchParams.get("group_key") ?? "";
```

Lee el filtro de grupo.

Ejemplos:

```
?group_key=server-01 → state.group_key = "server-01"
sin group_key        → state.group_key = ""
```

Función `syncForm`

```
function syncForm() {
```

Define una función para actualizar el formulario HTML con los valores actuales de `state`.

Esto es importante porque primero se lee la URL y después se reflejan esos valores en los controles visuales.

Sincronizar `limit`

```
qs("limit").value = String(state.limit);
```

Busca el elemento con ID `limit` y le asigna el valor del estado.

`qs()` viene de `app.js` y equivale a:

```
document.getElementById("limit")
```

`String(state.limit)` convierte el número a texto, porque los valores de los formularios son cadenas.

Sincronizar `status`

```
qs("status").value = state.status;
```

Actualiza el desplegable de estado.

Si `state.status` es `"open"`, el selector mostrará `open`.

Si es cadena vacía, mostrará `( todos )`.

Sincronizar `group_key`

```
qs("group_key").value = state.group_key;
```

Actualiza el input de `group_key`.

Si la URL contiene:

```
?group_key=server-01
```

el campo mostrará:

server-01

Función `syncUrl`

```
function syncUrl() {
```

Define una función para actualizar la URL con los valores actuales del estado.

Esto permite que la barra de direcciones refleje filtros y paginación.

Llamada a `setQueryParams`

```
setQueryParams({  
  limit: state.limit,  
  offset: state.offset,  
  status: state.status,  
  group_key: state.group_key,  
});
```

Llama a una función definida en `app.js`.

Le pasa un objeto con los parámetros que deben aparecer en la URL.

`setQueryParams()` elimina los valores vacíos y mantiene los que tienen contenido.

Ejemplo:

```
state = {  
  limit: 50,  
  offset: 0,  
  status: "open",  
  group_key: ""  
}
```

URL resultante:

```
index.html?limit=50&offset=0&status=open
```

Función `renderRows`

```
function renderRows(alerts) {
```

Define una función que recibe una lista de alertas y las pinta en la tabla HTML.

Parámetro:

```
alerts → lista de alertas recibidas desde GET /alerts
```

Seleccionar cuerpo de tabla

```
const tb = qs("alertsTbody");
```

Busca el elemento:

```
<tbody id="alertsTbody">
```

Este es el contenedor donde se insertan las filas.

Vaciar tabla

```
tb.innerHTML = "";
```

Borra el contenido actual del cuerpo de tabla.

Esto es necesario antes de pintar nuevos resultados.

Comprobar lista vacía

```
if (!alerts || alerts.length === 0) {
```

Comprueba si no hay alertas.

Casos:

```
alerts es null/undefined  
alerts es []
```

Si ocurre, se muestra una fila de “Sin resultados”.

Crear fila vacía

```
const tr = document.createElement("tr");
```

Crea un elemento `<tr>`.

Un `<tr>` representa una fila de tabla.

Insertar mensaje sin resultados

```
tr.innerHTML = ` Sin resultados</td>`; | | | | | | | | |
```

Define el contenido HTML de la fila.

La celda usa:

```
colspan="9"
```

porque la tabla tiene 9 columnas.

Esto hace que el mensaje ocupe todo el ancho de la tabla.

Añadir fila a la tabla

```
tb.appendChild(tr);
```

Inserta la fila dentro del `tbody`.

Salir de la función

```
return;
```

Termina la función.

Si no hay resultados, no se ejecuta el bucle posterior.

Recorrer alertas

```
for (const a of alerts) {
```

Recorre cada alerta de la lista.

En cada vuelta, `a` representa una alerta.

Ejemplo de `a` :

```
{
  "id": 1,
  "rule_id": 2,
  "event_id": 15,
  "title": "Rule matched: Failed login auth",
  "group_key": "server-01",
  "status": "open",
  "created_at": "2026-01-15T12:00:00",
  "updated_at": "2026-01-15T12:00:00"
}
```

Crear fila para alerta

```
const tr = document.createElement("tr");
```

Crea una nueva fila de tabla para la alerta actual.

Construir HTML de la fila

```
tr.innerHTML = `
  <td class="mono">${a.id}</td>
  <td><span class="${statusBadgeClass(a.status)}">${a.status}</span></td>
  <td class="muted"></td>
  <td class="mono">${a.group_key ?? "-"}</td>
  <td>${escapeHtml(a.title)}</td>
  <td class="mono">${a.rule_id}</td>
  <td class="mono">${a.event_id}</td>
  <td class="mono">${fmtDate(a.created_at)}</td>
  <td><a class="btn" href="/alert.html?id=${encodeURIComponent(a.id)}">Ver</a></td>
`;
```

Genera las celdas HTML de la fila.

Celda ID

```
<td class="mono">${a.id}</td>
```

Muestra el ID de la alerta.

La clase `mono` usa fuente monoespaciada.

Celda estado

```
<td><span class="${statusBadgeClass(a.status)}">${a.status}</span></td>
```

Muestra el estado dentro de un `span` .

`statusBadgeClass(a.status)` devuelve la clase CSS para el badge.

Actualmente siempre devuelve:

```
badge
```

Estados posibles:

```
open
ack
closed
```

Celda severidad

```
<td class="muted">—</td>
```

Muestra un placeholder.

Esto es intencionado.

El endpoint usado por esta página es:

```
GET /alerts
```

que devuelve `AlertOut`.

`AlertOut` no incluye severidad del evento.

Para mostrar severidad real habría que usar:

```
GET /alerts/ui
```

que devuelve `event_severity`.

Celda `group_key`

```
<td class="mono">${a.group_key ?? "—"}</td>
```

Muestra la clave de agrupación.

El operador:

```
??
```

usa `"—"` si `a.group_key` es `null` o `undefined`.

Celda título

```
<td>${escapeHtml(a.title)}</td>
```

Muestra el título de la alerta.

Se usa `escapeHtml()` para evitar insertar HTML sin controlar.

Esto es importante porque el valor viene de datos de la API.

Celda `rule_id`

```
<td class="mono">${a.rule_id}</td>
```

Muestra el ID de la regla que generó la alerta.

Celda `event_id`

```
<td class="mono">${a.event_id}</td>
```

Muestra el ID del evento que disparó la alerta.

Celda `created_at`

```
<td class="mono">${fmtDate(a.created_at)}</td>
```

Formatea la fecha de creación usando `fmtDate()` de `app.js`.

Celda enlace Ver

```
<td><a class="btn" href="/alert.html?id=${encodeURIComponent(a.id)}">Ver</a></td>
```

Crea un enlace hacia la página de detalle.

Ejemplo:

```
alert.html?id=1
```

`encodeURIComponent(a.id)` codifica el ID para usarlo de forma segura en una URL.

Añadir fila a tabla

```
tb.appendChild(tr);
```

Inserta la fila dentro del cuerpo de la tabla.

Función `escapeHtml`

```
function escapeHtml(s) {
```

Define una función para escapar caracteres especiales antes de insertar texto en HTML.

Es una medida básica de seguridad.

Convertir valor a string

```
return String(s ?? "")
```

Convierte el valor recibido a texto.

Si `s` es `null` o `undefined`, usa cadena vacía.

Escapar `&`

```
.replaceAll("&", "&amp;")
```

Reemplaza `&` por su entidad HTML.

Esto debe hacerse primero para evitar conflictos con otras entidades.

Escapar <

```
.replaceAll("<", "&lt;")
```

Evita que se interpreten etiquetas HTML de apertura.

Escapar >

```
.replaceAll(">", "&gt;")
```

Evita que se interpreten etiquetas HTML de cierre.

Escapar comillas dobles

```
.replaceAll('"', "&quot;")
```

Escapa comillas dobles.

Escapar comillas simples

```
.replaceAll("'", "&#039;");
```

Escapa comillas simples.

Con esto, un texto como:

```
<script>alert(1)</script>
```

se muestra como texto y no se ejecuta como HTML.

Función loadAlerts

```
async function loadAlerts() {
```

Define la función que carga alertas desde el backend.

Es asíncrona porque usa `await apiFetch(...)`.

Esta es la función principal del archivo.

Seleccionar cajas de mensaje

```
const err = qs("errorBox");  
const info = qs("infoBox");
```

Busca los elementos donde se mostrarán errores o mensajes informativos.

Ocultar mensajes anteriores

```
hide(err); hide(info);
```

Ocultar mensajes previos antes de hacer una nueva carga.

`hide()` viene de `app.js`.

Mostrar estado de carga

```
qs("alertsTbody").innerHTML = `<tr><td colspan="9" class="muted">Cargando...</td></tr>`;
```

Antes de pedir datos, muestra una fila de carga en la tabla.

Esto mejora la experiencia de usuario porque indica que la petición está en curso.

Bloque `try`

```
try {
```

Inicia un bloque para capturar errores durante la carga de alertas.

Si la API falla, se ejecutará el `catch`.

Llamada a la API

```
const data = await apiFetch("/alerts", {
  query: {
    limit: state.limit,
    offset: state.offset,
    status: state.status || null,
    group_key: state.group_key || null,
  }
});
```

Llama al endpoint:

```
GET /alerts
```

Pasando parámetros de consulta.

Query `limit`

```
limit: state.limit,
```

Envía el límite actual.

Ejemplo:

```
limit=50
```

Query `offset`

```
offset: state.offset,
```

Envía el desplazamiento actual.

Ejemplo:

```
offset=0
```

Query `status`

```
status: state.status || null,
```

Si `state.status` tiene valor, lo envía.

Si está vacío, envía `null`, y `apiFetch()` lo omitirá.

Ejemplo:

```
status=open
```

Query group_key

```
group_key: state.group_key || null,
```

Si hay `group_key`, lo envía.

Si está vacío, se omite.

Ejemplo:

```
group_key=server-01
```

Renderizar resultados

```
renderRows(data);
```

Pinta las alertas recibidas en la tabla.

`data` debe ser una lista de alertas devuelta por FastAPI.

Actualizar metadatos de resultado

```
qs("resultMeta").textContent =  
`limit=${state.limit} · offset=${state.offset} · resultados=${data.length}`;
```

Actualiza el texto que aparece debajo del título "Listado".

Ejemplo:

```
limit=50 · offset=0 · resultados=12
```

Esto ayuda a saber cuántos resultados se han cargado y en qué página se está.

Mensaje si no hay más resultados

```
if (data.length === 0 && state.offset > 0) {  
  show(info, "No hay más resultados en esta página. Prueba con Prev.");  
}
```

Si la API devuelve cero resultados y el usuario no está en la primera página, se muestra un mensaje.

Esto puede ocurrir al pulsar `Next` más allá del último bloque de resultados.

Estado del botón Prev

```
qs("btnPrev").disabled = state.offset <= 0;
```

Desactiva el botón `Prev` si ya se está en la primera página.

Si `offset` es 0, no tiene sentido retroceder.

Estado del botón Next

```
qs("btnNext").disabled = false;
```

Mantiene el botón Next habilitado.

El comentario explica que no se conoce el total de resultados, por lo que no se puede saber si hay siguiente página.

Por eso se permite avanzar y, si no hay datos, se muestra el mensaje informativo.

Captura de errores

```
} catch (e) {  
  show(err, `Error cargando alertas: ${e.message}`);  
}
```

Si algo falla, se muestra un error.

Ejemplos de fallos:

```
backend apagado  
CORS mal configurado  
endpoint no disponible  
error 500
```

El mensaje se muestra dentro de `errorBox`.

Función `init`

```
function init() {
```

Define la función de inicialización de la página.

Se ejecuta cuando el DOM ya está cargado.

Leer estado desde URL

```
readStateFromUrl();
```

Carga `limit`, `offset`, `status` y `group_key` desde la URL.

Sincronizar formulario

```
syncForm();
```

Rellena los controles del formulario con los valores del estado.

Esto hace que la interfaz refleje lo que hay en la URL.

Evento submit del formulario

```
qs("filtersForm").addEventListener("submit", (ev) => {
```

Añade un listener al formulario de filtros.

Cuando el usuario pulsa `Aplicar`, se ejecuta esta función.

Evitar recarga de página

```
ev.preventDefault();
```

Evita el comportamiento normal del formulario, que sería recargar la página.

Como la carga se hace por JavaScript, no interesa una recarga completa.

Leer estado del formulario

```
state.status = qs("status").value.trim();
state.group_key = qs("group_key").value.trim();
state.limit = Number(qs("limit").value);
```

Lee los valores actuales de los controles.

`trim()` elimina espacios al principio y al final.

Resetear paginación

```
state.offset = 0; // aplicar filtros resetea paginación
```

Cuando se aplican filtros nuevos, se vuelve a la primera página.

Esto evita quedarse en un `offset` alto que podría devolver cero resultados.

Sincronizar URL y cargar

```
syncUrl();
loadAlerts();
```

Primero actualiza la URL.

Después recarga alertas con los nuevos filtros.

Evento del botón Limpiar

```
qs("btnClear").addEventListener("click", () => {
```

Añade un listener al botón `Limpiar`.

Limpiar filtros

```
state.status = "";
state.group_key = "";
state.offset = 0;
```

Vacía los filtros y vuelve a la primera página.

El `limit` se conserva.

Actualizar formulario, URL y datos

```
syncForm();
syncUrl();
loadAlerts();
```

Refleja los cambios visualmente, actualiza la URL y recarga datos.

Evento botón Prev

```
qs("btnPrev").addEventListener("click", () => {
```

Añade lógica al botón de página anterior.

Calcular offset anterior

```
state.offset = Math.max(0, state.offset - state.limit);
```

Resta `limit` al `offset`.

`Math.max(0, ...)` evita que el `offset` sea negativo.

Ejemplo:

```
offset=50, limit=50 → offset=0  
offset=0, limit=50 → offset=0
```

Sincronizar y recargar

```
syncUrl();  
loadAlerts();
```

Actualiza la URL y recarga la tabla.

Evento botón Next

```
qs("btnNext").addEventListener("click", () => {
```

Añade lógica al botón de página siguiente.

Calcular offset siguiente

```
state.offset = state.offset + state.limit;
```

Suma `limit` al `offset`.

Ejemplo:

```
offset=0, limit=50 → offset=50
```

Sincronizar y recargar

```
syncUrl();  
loadAlerts();
```

Actualiza la URL y pide la siguiente página al backend.

Evento botón Actualizar

```
qs("btnRefresh").addEventListener("click", () => loadAlerts());
```

Asocia el botón `Actualizar` con una nueva carga de alertas.

No modifica filtros ni URL.

Solo vuelve a consultar la API.

Primera carga

```
loadAlerts();
```

Carga alertas al inicializar la página.

Esto hace que la tabla se rellene automáticamente al abrir `index.html`.

Ejecutar al cargar DOM

```
document.addEventListener("DOMContentLoaded", init);
```

Espera a que el HTML esté completamente cargado antes de ejecutar `init()`.

Esto es importante porque `init()` busca elementos del DOM como:

```
filtersForm  
btnClear  
btnPrev  
btnNext  
btnRefresh
```

Si se ejecutara antes de que existan, fallaría.

7 Relación con el flujo técnico del laboratorio

`alerts.js` conecta la página principal con el backend.

Flujo completo:

```
Usuario abre index.html  
↓  
DOMContentLoaded  
↓  
init()  
↓  
readStateFromUrl()  
↓  
syncForm()  
↓  
loadAlerts()  
↓  
apiFetch("/alerts")  
↓  
GET /alerts en FastAPI  
↓  
respuesta JSON con AlertOut  
↓  
renderRows()  
↓  
tabla HTML actualizada
```

Cuando el usuario aplica filtros:

```
Formulario filtros
  ↓
submit
  ↓
actualiza state
  ↓
syncUrl()
  ↓
GET /alerts con query params
  ↓
tabla filtrada
```

Cuando el usuario pulsa Ver :

```
Fila de tabla
  ↓
enlace alert.html?id=X
  ↓
página de detalle
```

8 Relación con backend

Este archivo consume principalmente:

```
GET /alerts
```

con estos parámetros:

```
limit
offset
status
group_key
```

Relación con `alerts.py` del backend:

```
alerts.js
  ↓
apiFetch("/alerts", { query })
  ↓
backend/app/api/routes/alerts.py
  ↓
list_alerts()
  ↓
select(Alert)
  ↓
PostgreSQL
  ↓
list[AlertOut]
```

La respuesta usada por `alerts.js` contiene campos de `AlertOut` :

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

Por eso la severidad aparece como placeholder.

9 Errores típicos o puntos importantes

El frontend actual no usa `/alerts/ui`

El listado usa:

```
apiFetch("/alerts", ...)
```

Por tanto, no recibe:

```
event_severity
event_source
event_message
rule_name
```

Para mostrar severidad real habría que cambiar a:

```
GET /alerts/ui
```

`escapeHtml()` protege el título

El campo `title` se inserta con:

```
escapeHtml(a.title)
```

Esto evita que contenido HTML se interprete como código.

Los IDs del HTML deben coincidir

Este archivo usa IDs definidos en `index.html`:

```
alertsTbody
errorBox
infoBox
resultMeta
btnPrev
btnNext
btnRefresh
filtersForm
btnClear
status
group_key
limit
```

Si alguno cambia en HTML, este archivo debe actualizarse.

La paginación no conoce el total

El backend básico `/alerts` no devuelve total.

Por eso `btnNext` siempre queda habilitado.

Si la página siguiente viene vacía, se muestra un aviso.

Una mejora sería usar un endpoint de conteo o consumir `/alerts/ui/count`.

`offset` se resetea al aplicar filtros

Esto evita que un filtro nuevo mantenga una página avanzada que no tenga resultados.

`syncUrl()` no recarga la página

Solo actualiza la URL.

Por eso después se llama manualmente a `loadAlerts()`.

`Number(...)` puede producir `NaN`

Si alguien modifica la URL manualmente:

```
?limit=abc
```

`Number("abc")` produce `NaN`.

Para este MVP no se valida en frontend, pero el backend sí tiene validación de `limit`.

Una versión más robusta podría normalizar esos valores.

10 Posibles mejoras futuras

Consumir `/alerts/ui`

Cambiar:

```
apiFetch("/alerts", ...)
```

por:

```
apiFetch("/alerts/ui", ...)
```

permitiría mostrar:

```
event_severity
event_source
event_message
rule_name
```

Usar `/alerts/ui/count`

Permitiría saber si existe siguiente página.

Flujo mejorado:

```
GET /alerts/ui
GET /alerts/ui/count
```

Así se podría desactivar `Next` al llegar al final.

Añadir más filtros

El backend `/alerts/ui` soporta:

```
severity_min
severity_max
source
q
rule_id
```

El frontend actual solo usa:

```
status
group_key
limit
offset
```

Añadir estilos por estado

`statusBadgeClass()` podría devolver clases diferentes:

```
badge badge-open
badge badge-ack
badge badge-closed
```

1 1 Comandos útiles relacionados

Servir frontend:

```
cd ~/siem-lab/frontend
python3 -m http.server 5173
```

Abrir listado:

```
http://localhost:5173/index.html
```

Abrir listado con filtros:

```
http://localhost:5173/index.html?limit=50&offset=0&status=open
```

Probar endpoint usado por el listado:

```
curl "http://localhost:8000/alerts?limit=50&offset=0"
```

Probar filtro por estado:

```
curl "http://localhost:8000/alerts?status=open"
```

Probar filtro por `group_key` :

```
curl "http://localhost:8000/alerts?group_key=server-01"
```

Probar endpoint enriquecido alternativo:

```
curl "http://localhost:8000/alerts/ui?status=open"
```

Abrir detalle de alerta:

```
http://localhost:5173/alert.html?id=1
```

Abrir consola del navegador:

```
F12 → Console
```

Comprobar errores habituales:

```
Failed to fetch
CORS error
```

404 Alert not found

500 Internal Server Error